

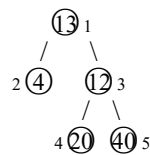
第五部分 图

5.1 医院设置

源程序名	hospital.??? (pas, c, cpp)
可执行文件名	hospital.exe
输入文件名	hospital.in
输出文件名	hospital.out

【问题描述】

设有一棵二叉树，如图 5-1：



其中，圈中的数字表示结点中居民的人口。圈边上数字表示结点编号，现在要求在某个结点上建立一个医院，使所有居民所走的路程之和为最小，同时约定，相邻接点之间的距离为1。如上图，若医院建在：

【输入】

第一行一个整数 n，表示树的结点数。(n≤100)

接下来的 n 行每行描述了一个结点的状况，包含三个整数，整数之间用空格(一个或多个)分隔，其中：第一个数为居民人口数；第二个数为左链接，为 0 表示无链接；第三个数为右链接。

【输出】

一个整数，表示最小距离和。

【样例】

hospital.in	hospital.out
5	81
13 2 3	
4 0 0	
12 4 5	
20 0 0	
40 0 0	

【知识准备】

图的遍历和最短路径。

【算法分析】

本题的求解任务十分明了：求一个最小路径之和。

根据题意，对 n 个结点，共有 n 个路径之和：用记号 Si 表示通向结点 i 的路径之和，

则 $S_i = \sum_{j=1}^i W_j g(i, j)$ ，其中 Wj 为结点 j 的居民数，g(i,j) 为结点 j 到结点 i 的最短路径长度。

下面表中反映的是样例的各项数据：

$g(i,j)$	j	1	2	3	4	5	Si
i		0	1	1	2	2	$0 \times 13 + 1 \times 4 + 1 \times 12 + 2 \times 20 + 2 \times 40 = 136$

2	1	0	2	3	3	$1 \times 13 + 0 \times 4 + 2 \times 12 + 3 \times 20 + 3 \times 40 = 217$
3	1	2	0	1	1	$1 \times 13 + 2 \times 4 + 0 \times 12 + 1 \times 20 + 1 \times 40 = 81$
4	2	3	1	0	2	$2 \times 13 + 3 \times 4 + 1 \times 12 + 0 \times 20 + 2 \times 40 = 130$
5	2	3	1	2	0	$2 \times 13 + 3 \times 4 + 1 \times 12 + 2 \times 20 + 0 \times 40 = 90$

从表中可知 S3=81 最小，医院应建在 3 号居民点，使得所有居民走的路径之和为最小。

由此可知，本题的关键是求 $g[i,j]$ ，即图中任意两点间的最短路径长度。

求任意两点间的最短路径采用下面的弗洛伊德（Floyd）算法。

（1）数据结构：

Int w[101]; 描述个居民点人口数

g[101][101]; 初值为图的邻接矩阵，最终为最短路径长度

（2）数据的读入：

本题数据结构的原形为二叉树，数据提供为孩子标识法，分支长度为 1，建立带权图的邻接矩阵，分下面两步：

① $g[i,j] \leftarrow \text{Max}$ {Max 为一较大数，表示结点 i 与 j 之间无直接相连边}

② 读入 n 个结点信息：

```
for(i=1;;<=n;i++)
```

```
{
```

```
    g[i][j]=0;
```

```
    cin>>w[i]>>l>>r;
```

```
    if(l>0) {
```

```
        g[i][l]=1;        g[l][i]=1
```

```
    }
```

```
    if(r>0) {
```

```
        g[i][r]=1;        g[r][i]=1
```

```
    }
```

（3）弗洛伊德算法求任意两点间的最短路径长度

```
for(k=1;k<=n;k++)
```

```
    for(i=1;i<=n;i++)
```

```
        if(i!=k) for(j=1;j<=n;j++)
```

```
            if( (i!=j) && (k!=j) && (g[i][k]+g[k][j]<g[i][j]) ) g[i][j]=g[i][k]+g[k][j];
```

（4）求最小的路程和 min

```
min=max longint;
```

```
for(i=1;i<=n;i++)
```

```
{
```

```
    sum=0;
```

```
    for(j=1;j<=n;j++) sum=sum+w[i]*g[i][j];
```

```
    if(sum<min) min=sum;
```

```
}
```

（5）输出

```
Cout<<min;
```

5.2 工程规划

源程序名	work.??? (pas, c, cpp)
可执行文件名	work.exe
输入文件名	work.in
输出文件名	work.out

【问题描述】

造一幢大楼是一项艰巨的工程，它是由 n 个子任务构成的，给它们分别编号 $1, 2, \dots, n$ ($5 \leq n \leq 1000$)。由于对一些任务的起始条件有着严格的限制，所以每个任务的起始时间 $T_1, T_2, \dots, T_n$ 并不是很容易确定的(但这些起始时间都是非负整数，因为它们必须在整个工程开始后启动)。例如：挖掘完成后，紧接着就要打地基；但是混凝土浇筑完成后，却要等待一段时间再去掉模板。

这种要求就可以用 M ($5 \leq m \leq 5000$) 个不等式表示，不等式形如 $T_i - T_j \leq b$ 代表 i 和 j 的起始时间必须满足的条件。每个不等式的右边都是一个常数 b ，这些常数可能不相同，但是它们都在区间 $(-100, 100)$ 内。

你的任务就是写一个程序，给定像上面那样的不等式，找出一种可能的起始时间序列 $T_1, T_2, \dots, T_n$ ，或者判断问题无解。对于有解的情况，要使最早进行的那个任务和整个工程的起始时间相同，也就是说， $T_1, T_2, \dots, T_n$ 中至少有一个为 0。

【输入】

第一行是用空格隔开的两个正整数 n 和 m ，下面的 m 行每行有三个用空格隔开的整数 i, j, b 对应着不等式 $T_i - T_j \leq b$ 。

【输出】

如果有可行的方案，那么输出 N 行，每行都有一个非负整数且至少有一个为 0，按顺序表示每个任务的起始时间。如果没有可行的方案，就输出信息 “NO SOLUTION”。

【样例 1】

work.in	work.out
5 8	0
1 2 0	2
1 5 -1	5
2 5 1	4
3 1 5	1
4 1 4	
4 3 -1	
5 3 -1	
5 4 -3	

【样例 2】

work.in	work.out
5 5	NO SOLUTION
1 2 -3	
1 5 -1	
2 5 -1	
5 1 -5	
4 1 4	

【算法分析】

本题是一类称为约束满足问题的典型问题，问题描述成 n 个子任务的起始时间 T_i 及它们之间在取值上的约束，求一种满足所有约束的取值方法。

将工程的 n 个子任务 $1, 2, \dots, n$ 作为一有向图 G 的 n 个顶点，顶点 V_i ($i=1, \dots, n$) 的关键值为子任务 i 的起始时间 T_i ，我们并不需要关心顶点之间的弧及其弧长，而是要确定顶点的关键值 T_i 的取值，满足所有的约束条件。本题的约束条件由 m 个不等式 $T_i - T_j \leq b$ 给出，这样的有向图称为约束图。

为了确定每一个 T_i 的值，先假设某一个子任务的起始时间为零，如设 $T_j=0$ ，则其余子任务的起始时间 T_i 相对于 T_j 可设置其起始时间为一区间 $[-\max t, \max t]$ 。

下面分析不等式 $T_i - T_j \leq b$ 。此不等式可变形为如下两种形式：

(1) $T_i \leq T_j + b$ 意味 T_i 的最大值为 $T_j + b$ ；

(2) $T_j \geq T_i - b$ 意味 T_j 的最大值为 $T_i - b$ ；

因此，根据题中给出的 m 个不等式，逐步调整各个 T_i 的最小值和最大值。

设 $\text{high}[i]$ 为 T_i 当前的最大值， $\text{low}[i]$ 为 T_i 当前的最小值。

$\text{high}[j]$ 为 T_j 当前的最大值， $\text{low}[j]$ 为 T_j 当前的最小值。

若 $\text{high}[i] - \text{high}[j] > b$ ，则 $\text{high}[i] = \text{high}[j] + b$ (根据 $T_i \leq T_j + b$)，

若 $\text{low}[i] - \text{low}[j] < b$ ，则 $\text{low}[j] = \text{low}[i] - b$ (根据 $T_i \geq T_j - b$)。

以上的调整终止视下列两种情况而定：

(1) 对所有的不等式 $T_i - T_j \leq b$ ，不再有 $\text{high}[i]$ 或 $\text{low}[j]$ 的调整；

(2) 若存在 $\text{high}[i] < \text{low}[i]$ 或 $\text{high}[j] < \text{low}[j]$ 则表示不存在 $T_1, T_2, \dots, T_n$ 能满足所有 m 个不等式 $T_i - T_j \leq b$ ，即问题无解。

根据以上思路，先建立约束图，每个结点代表一个起始时间值，并记录其可能的取值范围：

数组 $\text{int high}[\max n], \text{low}[\max n]$ ； $\{n$ 个子任务起始时间的取值范围 $\}$

$\text{high}[1]=0; \quad \text{low}[1]=0; \quad \{\text{设置 } n \text{ 个起始时间的初值，其中 } T_i=0\}$

for($i=2; i \leq n; i++$) {

$\text{high}[i]=\max t; \quad \{T_i \text{ 的上界}\}$

$\text{low}[i]=-\max t; \quad \{T_j \text{ 的下界}\}$

}

约束条件 (m 个不等式) 用记录数组表示：

struct {不等式结构}

 Tinequ{

 int i, j, b ;

 }

 Tinequ arrinequ[$\max m$]; {存放 m 个不等式}

利用约束条件，逐一调整每一个起始时间的取值范围，直到无法调整为止。

主要算法如下：

```
flag=true;           {调整状态标记}
noans=false;         {解的有无标记}
while (flag)         {进行约束传递，根据不等式调整各个起始时间值}
{
    flag=false;
    for(k=1;k<=m;k++)
        讨论 arrinequ[k]
    {
```

```
if( (high[i]-high[j]>b) ) { high[i]=high[j]+b;  flag=true;  }   {调整 Ti 的上界}
if( (low[i]-low[j]>b) ) { low[j]=low[i]-b;   flag=true;  }   {调整 Tj 的下界}
if( (low[i]>high[i]) or (low[j]>high[j])
) {
    {无法满足当前不等式，则调整终止}
    noans=true;    {问题无解 noans=true}
    flag=false;
    break;
}
}
```

下面以样例说明：

【样例 1】

8 个不等式如下

序号	1	2	3	4	5	6	7	8
i	1	1	2	3	4	4	5	5
j	2	5	5	1	1	3	3	4
b	0	-1	1	5	4	-1	-3	-3

顶点的键值 Ti 的调整记录：

	初值	第 1 轮调整	第 2 轮调整	第 3 轮调整
high[1]	0	0	0	0
low[1]	0	0	0	0
high[2]	100000	100000	2	2
low[2]	-10000	2	2	2
high[3]	100000	5	5	5
low[3]	-10000	4	5	5
high[4]	100000	4	4	4
low[4]	-10000	4	4	4
high[5]	100000	1	1	1
low[5]	-10000	1	1	1
调整状态		有变化	有变化	无变化

【样例 2】

5 个不等式如下

编号	1	2	3	4	5
i	1	1	2	5	4
j	2	5	5	1	1
b	-3	-1	-1	-5	4

顶点键值 Ti 的调整记录：

		初值	第一轮调整	第二轮调整
1	high	0	0	
	low	0	0	
2	high	100000	99999	
	low	-10000	3	
3	high	100000	10000	
	low	-10000	-10000	
4	high	100000	4	
	low	-10000	-10000	
5	high	100000	-5	

	low	-10000	1	
调整情况			high[5]<low[5]终止	

经第一轮调整，调整过程终止，即问题无解。

从样例 2 所给不等式也可看出，因为：

$$T_1 - T_2 \leq -3, \rightarrow T_2 > T_1$$

$$T_2 - T_5 \leq -1, \rightarrow T_5 > T_2$$

$$T_5 - T_1 \leq -5, \rightarrow T_1 > T_5$$

这三个不等式不能同时成立，因此问题无解。

5.3 服务器储存信息问题

源程序名	servers.??? (pas, c, cpp)
可执行文件名	servers.exe
输入文件名	servers.in
输出文件名	servers.out

【问题描述】

Bytel&&王国准备在各服务器间建立大型网络并提供多种服务。

网络由 n 台服务器组成，用双向的线连接。两台服务器之间最多只能有一条线直接连接，同时，每台服务器最多只能和 10 台服务器直接连接，但是任意两台服务器间必然存在一条路径将它们连接在一起。每条传输线都有一个固定传输的速度。 $\delta(V, W)$ 表示服务器 V 和 W 之间的最短路径长度，且对任意的 V 有 $\delta(V, V) = 0$ 。

有些服务器比别的服务器提供更多的服务，它们的重要程度要高一些。我们用 $r(V)$ 表示服务器 V 的重要程度(rank)。rank 越高的服务器越重要。

每台服务器都会存储它附近的服务器的信息。当然，不是所有服务器的信息都存，只有感兴趣的服务器信息才会被存储。服务器 V 对服务器 W 感兴趣是指，不存在服务器 U 满足， $r(U) > r(W)$ 且 $\delta(V, U) < \delta(V, W)$ 。

举个例子来说，所有具有最高 rank 的服务器都会被别的服务器感兴趣。如果 V 是一台具有最高 rank 的服务器，由于 $\delta(V, V) = 0$ ，所以 V 只对具有最高 rank 的服务器感兴趣。我们定义 $B(V)$ 为 V 感兴趣的服务器的集合。

我们希望计算所有服务器储存的信息量，即所有服务器的 $|B(V)|$ 之和。Bytel&&王国并不希望存储大量的数据，所以所有服务器存储的数据量 ($|B(V)|$ 之和) 不会超过 $30n$ 。

你的任务是写一个程序，读入 Bytel&&王国的网络分布，计算所有服务器存储的数据量。

【输入】

第一行两个整数 n 和 m ，($1 \leq n \leq 30000$ ， $1 \leq m \leq 5n$)。 n 表示服务器的数量， m 表示传输线的数量。

接下来 n 行，每行一个整数，第 i 行的整数为 $r(i)$ ($1 \leq r(i) \leq 10$)，表示第 i 台服务器的 rank。

接下来 m 行，每行表示各条传输线的信息，包含三个整数 a ， b ， t ($1 \leq t \leq 1000$ ， $1 \leq a$ ， $b \leq n$ ， $a \neq b$)。 a 和 b 是传输线所连接的两台服务器的编号， t 是传输线的长度。

【输出】

一个整数，表示所有服务器存储的数据总量，即 $|B(V)|$ 之和。

【样例】

servers.in servers.out

2

3

1

1

1 4 30

2 3 20

3 4 20

注: $B(1)=\{1, 2\}$, $B(2)=\{2\}$, $B(3)=\{2, 3\}$, $B(4)=\{1, 2, 3, 4\}$ 。

【知识准备】

Dijkstra 算法, 及其 $O((n+e)\log_2 n)$ 或 $O(n\log_2 n+e)$ 的实现。

【算法分析】

本题的难点在于问题的规模。如果问题的规模在 100 左右, 那么这将是一道非常容易的题目。因为 $O(n^3)$ 的算法是很容易想到的:

(1) 求出任意两点间的最短路径, 时间复杂度为 $O(n^3)$;

(2) 枚举任意两点, 根据定义判断一个节点是否对另一个节点感兴趣, 时间复杂度为 $O(n^3)$ 。

当然, 对于 30000 规模的本题来说, $O(n^3)$ 的算法是绝对不可行的, 即便降到 $O(n^2)$ 也不行, 只有 $O(n\log_2 n)$ 或 $O(n)$ 是可以接受的。

既然现在可以得到的算法与要求相去甚远, 要想一鼓作气得到一个可行的算法似乎就不是那么容易了。我们不妨先来看看我们可以做些什么。

判断一个节点 V 是否对节点 W 感兴趣, 就是要判断是否存在一个 rank 大于 $r(W)$ 的节点 U , $\delta(V, U) < \delta(V, W)$ 。所以, 节点 V 到某个特定的 rank 的节点 (集合) 的最短距离是一个非常重要的值。如果我們可以在 $O(n\log_2 n)$ 时间内求出所有节点到特定 rank 的节点 (集合) 的最短距离, 我们就成功地完成了算法的一个重要环节。

用 Dijkstra 算法反过来求特定 rank 的节点 (集合) 到所有点的最短距离——以所有特定 rank 的节点为起点 ($\text{rank}=1, 2, 3, \dots$ 或 10), 求这一点集到所有点的最短距离。由于图中与每个点关联的边最多只有 10 条, 所以图中的边数最多为 $5n$ 。用 Priority Queue (Heap, Winner Tree 或 Fibonacci Heap 等) 来实现 Dijkstra 算法, 时间复杂度为 $O((n+e)\log_2 n)$ (用 Fibonacci Heap 实现, 更确切的时间复杂度是 $O(n\log_2 n+e)$)。这里, $e=5n$, 因而求一遍最短路径的时间复杂度为 $O(n\log_2 n)$ 。由于 $1 \leq \text{rank} \leq 10$, 每个 rank 都要求一遍最短路径, 所以求出每个节点到所有 rank 的最短路径长度的时间复杂度为 $O(10 \cdot (5+1)n\log_2 n)$, 即 $O(n\log_2 n)$ 。

求出所有点到特定 rank 的节点 (集合) 的最短距离, 就完成了判断任意节点 V 对 W 是否感兴趣的一半工作。另一半是求任意节点 V 到 W 的最短距离。前面求节点到 rank 的最短距离时, 利用的是 rank 范围之小——只有 10 种, 以 10 个 rank 集合作起点, 用 Dijkstra 算法求 10 次最短路径。但是, 如果是求任意两点的最短路径, 就不可能只求很少次数的最短路径了。一般来说, 求任意两点最短路径是 $\Omega(n^2)$ 的 (这只是一个很松的下界), 这样的规模已经远远超出了可承受的范围。但是, 要判断 V 对 W 是否感兴趣, $\delta(V, W)$ 又是必须求的, 所以 n 次 Dijkstra 算法求最短路径肯定是逃不掉的 (或者也可以用一次 Floyd 算法代替, 但是时间复杂度一样, 可认为等价)。那么, 我们又能从哪里来降这个时间复杂度呢?

题目中提到: 所有服务器储存的数据量 ($|B(V)|$ 之和) 不会超过 $30n$ 。这就是说, 最多只存在 $30n$ 对 (V, W) 满足 V 对 W 感兴趣。所以, 就本题来说, 我们需要处理的点对最少可能只有 $30n$ 个, 求最短距离的下界也就变成 $\Omega(30n) = \Omega(n)$ 了 (当然, 这也只是很松的下界)。虽说下界是 $\Omega(n)$, 其实我们只需要有 $O(n\log_2 n)$ 的算法就可以满足要求了。

从前面估算下界的过程中我们也看到，计算在下界中的代价都是感兴趣的点对（一个节点对另一个节点感兴趣），其余部分为不感兴趣的点对。我们如果想降低时间复杂度，就要避免不必要的计算，即避免计算不感兴趣的点对的最短路径。

我们来看当 V 对 W 不感兴趣时的情况。根据定义， $\delta(V, W) > \delta(V, r(W)+1)$ 。如果是以 W 为起点，用 Dijkstra 算法求最短路径的话。当扩展到 V 时，发现 V 对 W 不感兴趣，即 $\delta(V, W) > \delta(V, r(W)+1)$ 。那么，如果再由 V 扩展求得到 U 的最短路径，则：

$$\delta(U, W) = \delta(V, W) + \delta(U, V),$$

$$\delta(U, r(W)+1) = \delta(V, r(W)+1) + \delta(U, V),$$

$$\text{由于 } \delta(V, W) > \delta(V, r(W)+1),$$

$$\text{所以 } \delta(V, W) + \delta(U, V) > \delta(V, r(W)+1) + \delta(U, V), \text{ 即 } \delta(U, W) > \delta(U, r(W)+1)$$

所以， U 对 W 也不感兴趣。因此，如果以 W 为起点，求其他点到 W 的最短路径，以判断其他点是否对 W 感兴趣，当扩展到对 W 不感兴趣的节点时，就可以不继续扩展下去了（只扩展对 W 感兴趣的节点）。

我们知道，所有感兴趣的点对不超过 $30n$ 。因此，以所有点作起点，用 Dijkstra 算法求最短路径的时间复杂度可由平摊分析得为 $O(30(n+e) \log_2 n) = O(30(n+5n) \log_2 n) = O(n \log_2 n)$ 。

由此，我们看到判断一节点是否对另一节点感兴趣，两个关键的步骤都可以在 $O(n \log_2 n)$ 时间内完成。当然算法的系数是很大的，不过由于 n 不大，这个时间复杂度还是完全可以承受的。下面就总结一下前面得到的算法：

（1）分别以 $\text{rank}=1, 2, \dots, 10$ 的节点（集合）作为起点，求该节点（集合）到所有点的最短距离（其实也就是所有点到该节点（集合）的最短距离）；

（2）以每个点作为起点，求该点到所有点的最短距离。当求得某节点的最短距离的同时根据求得的最短距离和该节点到 rank 大于起点的节点（集合）的最短距离，判断该节点是否对起点感兴趣。如果感兴趣，则找到一对感兴趣的点对，否则，停止扩展该节点，因为该节点不可能扩展出对起点感兴趣的节点。

总结解题的过程，可以发现解决本题的关键有三点：一是稀疏图，正因为图中边比较稀疏所以我们可以用 Dijkstra+Priority Queue 的方法将求最短路径的时间复杂度降为 $O(n \log_2 n)$ ；二是 rank 的范围很小， rank 的范围只有 10，所以我们只用了 10 次 Dijkstra 算法就求得了所有点到特定 rank 的最短距离；三是感兴趣的点对只有很少，由于感兴趣的点对只有 $30n$ ，我们通过只计算感兴趣点对的最短路径，将求点与点间最短路径的时间复杂度降到了 $O(n \log_2 n)$ 。这三点，只要有一点没有抓住。本题就不可能得到解决。

5.4 间谍网络(AGE)

源程序名	age.??? (pas, c, cpp)
可执行文件名	age.exe
输入文件名	age.in
输出文件名	age.out

【问题描述】

由于外国间谍的大量渗入，国家安全正处于高度的危机之中。如果 A 间谍手中掌握着关于 B 间谍的犯罪证据，则称 A 可以揭发 B。有些间谍收受贿赂，只要给他们一定数量的美元，他们就愿意交出手中掌握的全部情报。所以，如果我们能够收买一些间谍的话，我们就可能控制间谍网中的每一分子。因为一旦我们逮捕了一个间谍，他手中掌握的情报都将归我们所有，这样就有可能逮捕新的间谍，掌握新的情报。

我们的反间谍机关提供了一份资料，包括所有已知的受贿的间谍，以及他们愿意收受的具体数额。同时我们还知道哪些间谍手中具体掌握了哪些间谍的资料。假设总共有 n 个间谍 (n 不超过 3000)，每个间谍分别用 1 到 3000 的整数来标识。

请根据这份资料，判断我们是否有可能控制全部的间谍，如果可以，求出我们需要支付的最少资金。否则，输出不能被控制的一个间谍。

【输入】

输入文件 age.in 第一行只有一个整数 n。

第二行是整数 p。表示愿意被收买的人数， $1 \leq p \leq n$ 。

接下来的 p 行，每行有两个整数，第一个数是一个愿意被收买的间谍的编号，第二个数表示他将会被收买的数额。这个数额不超过 20000。

紧跟着一行只有一个整数 r， $1 \leq r \leq 8000$ 。然后 r 行，每行两个正整数，表示数对 (A, B)，A 间谍掌握 B 间谍的证据。

【输出】

答案输出到 age.out。

如果可以控制所有间谍，第一行输出 YES，并在第二行输出所需要支付的贿金最小值。否则输出 NO，并在第二行输出不能控制的间谍中，编号最小的间谍编号。

【样例 1】

age.in	age.out
3	YES
2	110
1 10	
2 100	
2	
1 3	
2 3	

【样例 2】

age.in	age.out
4	NO
2	3
1 100	
4 200	
2	
1 2	
3 4	

【算法分析】

根据题中给出的间谍的相互控制关系，建立有向图。找出有向图中的所有强连通分量，

用每个强连通分量中最便宜的点（需支付最少贿金的间谍）来代替这些强连通分量，将强连通分量收缩为单个节点。收缩强连通分量后的图中，入度为 0 的节点即代表需要贿赂的间谍。

5.5 宫廷守卫

源程序名	guards.??? (pas, c, cpp)
可执行文件名	guards.exe
输入文件名	guards.in
输出文件名	guards.out

【问题描述】

从前有一个王国，这个王国的城堡是一个矩形，被分为 $M \times N$ 个方格。一些方格是墙，而另一些是空地。这个王国的国王在城堡里设了一些陷阱，每个陷阱占据一块空地。

一天，国王决定在城堡里布置守卫，他希望安排尽量多的守卫。守卫们都是经过严格训练的，所以一旦他们发现同行或同列中有人的话，他们立即向那人射击。因此，国王希望能够合理地布置守卫，使他们互相之间不能看见，这样他们就不可能互相射击了。守卫们只能被布置在空地上，不能被布置在陷阱或墙上，且一块空地只能布置一个守卫。如果两个守卫在同一行或同一列，并且他们之间没有墙的话，他们就能互相看见。（守卫就像象棋里的车一样）

你的任务是写一个程序，根据给定的城堡，计算最多可布置多少个守卫，并设计出布置的方案。

【输入】

第一行两个整数 M 和 N ($1 \leq M, N \leq 200$)，表示城堡的规模。

接下来 M 行 N 列的整数，描述的是城堡的地形。第 i 行 j 列的数用 a_{ij} 表示。

$a_{ij}=0$ ，表示方格 $[i,j]$ 是一块空地；

$a_{ij}=1$ ，表示方格 $[i,j]$ 是一个陷阱；

$a_{ij}=2$ ，表示方格 $[i,j]$ 是墙。

【输出】

第一行一个整数 K ，表示最多可布置 K 个守卫。

此后 K 行，每行两个整数 x_i 和 y_i ，描述一个守卫的位置。

【样例】

guards.in	guards.out
3 4	2
2 0 0 0	1 2
2 2 2 1	3 3
0 1 0 2	

样例数据如图 5-2（黑色方格为墙，白色方格为空地，圆圈为陷阱，G 表示守卫）

■	□	□	□
■	■	■	○
□	○	□	■

■	G	□	□
■	■	■	○
□	○	G	■

【算法分析】

本题的关键在构图。

城堡其实就是一个棋盘。我们把棋盘上横向和纵向连续的极长段（不含墙）都分离出来。显然，每一段上最多只能放一个 guard，而且 guard 总是放在一个纵向段和一个横向段的交

界处，所以一个 **guard** 和一个纵向段和一个横向段有关。

我们把纵向段和横向段都抽象成图中的节点，如果一个纵向段和一个横向段相交的话，就在两点之间连一条边。这样，**guard** 就成为了图中的边。前面得出的性质抽象成图的语言就是，每个点只能和一条边相连，每条边只能连接一个纵向边的点和一个横向边的点。因此，这样的图是二分图，我们所求的正是二分图的匹配。而要布置最多的 **guards**，就是匹配数要最大，即最大匹配。

图中节点数为 n ($n \leq 200$)，求最大匹配的时间复杂度为 $O(n^{2.5})$ 。

5.6 K-联赛

源程序名	kleague.??? (pas, c, cpp)
可执行文件名	kleague.exe
输入文件名	kleague.in
输出文件名	kleague.out

【问题描述】

K-联赛职业足球俱乐部的球迷们都是有组织的训练有素的啦啦队员，就像红魔啦啦队一样(2002 年韩日世界杯上韩国队的啦啦队)。这个赛季，经过很多场比赛以后，球迷们希望知道他们支持的球队是否还有机会赢得最后的联赛冠军。换句话说，球队是否可以通过某种特定的比赛结果最终取得最高的积分(获胜场次最多)。(允许出现多支队并列第一的情况。)

现在，给出每个队的胜负场数， w_i 和 d_i ，分别表示 $team_i$ 的胜场和负场($1 \leq i \leq n$)。还给出 a_{ij} ，表示 $team_i$ 和 $team_j$ 之间还剩多少场比赛要进行($1 \leq i, j \leq n$)。这里， n 表示参加联赛的队数，所有的队分别用 1, 2, ..., n 来编号。你的任务是找出所有还有可能获得冠军的球队。

所有队参加的比赛数是相同的，并且为了简化问题，你可以认为不存在平局(比赛结果只有胜或负两种)。

【输入】

第一行一个整数 n ($1 \leq n \leq 25$)，表示联赛中的队数。

第二行 $2n$ 个数， $w_1, d_1, w_2, d_2, \dots, w_n, d_n$ ，所有的数不超过 100。

第三行 n^2 个数， $a_{1,1}, a_{1,2}, \dots, a_{1,n}, a_{2,1}, \dots, a_{2,2}, a_{2,n}, \dots, a_{n,1}, a_{n,2}, \dots, a_{n,n}$ ，所有的数都不超过 10。 $a_{ij}=a_{ji}$ ，如果 $i=j$ ，则 $a_{ij}=0$ 。

【输出】

仅一行，输出所有可能获得冠军的球队，按其编号升序输出，中间用空格分隔。

【样例 1】

kleague.in	kleague.out
3	1 2 3
2 0 1 1 0 2	
0 2 2 2 0 2 2 2 0	

【样例 2】

kleague.in	kleague.out
3	1 2
4 0 2 2 0 4	
0 1 1 1 0 1 1 1 0	

【样例 3】

kleague.in	kleague.out
4	2 4
0 3 3 1 1 3 3 0	
0 0 0 2 0 0 1 0 0 1 0 0 2 0 0 0	

【算法分析】

看一个队是否有希望夺冠，首先，这个队此后的比赛自然是赢越多越好，所以先让这个队把以后的比赛都赢下来，算出这个队最高能拿多少分。下面关键就看是否有可能让其他队的积分都低于刚才计算出的最高分。

建立一个网络，所有的球队作为图中的节点，每两个队之间的比赛也作为图中的节点。从网络的源各连一条边到“比赛的节点”，容量为两队间还剩的比赛场数。从“每个队的节点”都连一条边到网络的汇，容量为这个队当前的积分与最高分之差。如果一个“比赛的节点”代表的是 A 与 B 之间的比赛，那么从这个节点连两条边分别到“A 队的节点”和“B

队的节点”，容量为无穷大。

如果这个网络的最大流等于所有还未进行的比赛的场次之和，那么需要我们判断的那个队抗有可能夺得冠军。

本题要我们找出所有可能夺冠的队，那么只需枚举所有的球队，判断它们是否有可能夺冠即可。

5.7 机器调度

源程序名	machine.??? (pas, c, cpp)
可执行文件名	machine.exe
输入文件名	machine.in
输出文件名	machine.out

【问题描述】

我们知道机器调度是计算机科学中一个非常经典的问题。调度问题有很多种，具体条件不同，问题就不同。现在我们要处理的是两个机器的调度问题。

有两个机器 A 和 B。机器 A 有 n 种工作模式，我们称之为 $mode_0, mode_1, \dots, mode_n-1$ 。同样，机器 B 有 m 种工作模式，我们称之为 $mode_0, mode_1, \dots, mode_m-1$ 。

初始时，两台机器的工作模式均为 $mode_0$ 。现在有 k 个任务，每个工作都可以在两台机器中任意一台的特定的模式下被加工。例如， job_0 能在机器 A 的 $mode_3$ 或机器 B 的 $mode_4$ 下被加工， job_1 能在机器 A 的 $mode_2$ 或机器 B 的 $mode_4$ 下被加工，等等。因此，对于任意的 job_i ，我们可以用三元组 (i, x, y) 来表示 job_i 在机器 A 的 $mode_x$ 或机器 B 的 $mode_y$ 下被加工。

显然，要完成所有工作，我们需要不时的改变机器的工作模式。但是，改变机器的工作状态就必须重启机器，这是需要代价的。你的任务是，合理的分配任务给适当的机器，使机器的重启次数尽量少。

【输入】

第一行三个整数 n, m ($n, m < 100$), k ($k < 1000$)。接下来的 k 行，每行三个整数 i, x, y 。

【输出】

只一行一个整数，表示最少的重启次数。

【样例】

machine.in	machine.out
5 5 10	3
0 1 1	
1 1 2	
2 1 3	
3 1 4	
4 2 1	
5 2 2	
6 2 3	
7 2 4	
8 3 3	
9 4 3	

【问题分析】

本题所求的是工作模式的最少切换次数，实际上也就是求最少需要使用多少个工作模

式，因为一个工作模式被切换两次肯定是不合算的，一旦切换到一个工作模式就应该把这个工作模式可以完成的工作都完成。

将两台机器的工作模式分别看成 n 个和 m 个节点。 job_i 分别和机器 A 和 B 的 `mode_x` 和 `mode_y` 相关： job_i 要被完成，就必须切换到机器 A 的 `mode_x` 或切换到机器 B 的 `mode_y`。将 job_i 看作图中的一条边——连接节点 x 和节点 y 的边，那么这条边就要求 x 和 y 两个节点中至少要有有一个节点被取出来。这正符合覆盖集的性质。

我们构成的图是二分图，要求最少的切换次数，就是要使覆盖集最小。二分图的最小覆盖集问题等价于二分图的最大匹配问题。因此，只需对此二分图求一个最大匹配即是我们要求的答案。时间复杂度 $O((m+n)^{\frac{1}{2}}k)$ 。

5.8 公路修建

源程序名	road.??? (pas, c, cpp)
可执行文件名	road.exe
输入文件名	road.in
输出文件名	road.out

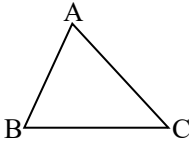
【问题描述】

某国有 n 个城市，它们互相之间没有公路相通，因此交通十分不便。为解决这一“行路难”的问题，政府决定修建公路。修建公路的任务由各城市共同完成。

修建工程分若干轮完成。在每一轮中，每个城市选择一个与它最近的城市，申请修建通往该城市的公路。政府负责审批这些申请以决定是否同意修建。

政府审批的规则如下：

- (1) 如果两个或以上城市申请修建同一条公路，则让它们共同修建；
- (2) 如果三个或以上的城市申请修建的公路成环。如下图，A 申请修建公路 AB，B 申请修建公路 BC，C 申请修建公路 CA。则政府将否决其中最短的一条公路的修建申请；



- (3) 其他情况的申请一律同意。

一轮修建结束后，可能会有若干城市可以通过公路直接或间接相连。这些可以互相：连通的城市即组成“城市联盟”。在下一轮修建中，每个“城市联盟”将被看作一个城市，发挥一个城市的作用。

当所有城市被组合成一个“城市联盟”时，修建工程也就完成了。

你的任务是根据城市的分布和前面讲到的规则，计算出将要修建的公路总长度。

【输入】

第一行一个整数 n ，表示城市的数量。 $(n \leq 5000)$

以下 n 行，每行两个整数 x 和 y ，表示一个城市的坐标。 $(-1000000 \leq x, y \leq 1000000)$

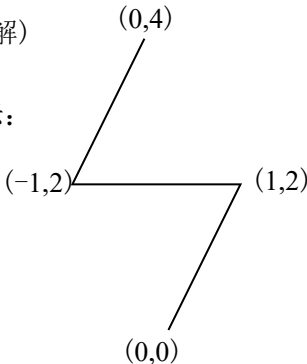
【输出】

一个实数，四舍五入保留两位小数，表示公路总长。(保证有惟一解)

【样例】

road.in	road.out
4	6.47
0 0	
1 2	

修建的公路如图所示：



-1 2

0 4

【问题分析】

三条规则中的第二条是故弄玄虚。如果三个或三个以上的城市申请修建的公路成环，那么这些公路的长度必然都相同，否则不满足“每个城市选择一个与它最近的城市，申请修建通往该城市的公路”。所以，如果成环，其实是任意去掉一条路。

本题要我们求的实际上是最小生成树，也就是说，按规则生成的是图的最小生成树。为什么呢？很显然，按规则生成的应该是树。根据规则：每个城市选择一个与它最近的城市，申请修建通往该城市的公路。那么，对于图中任意的环，环上最长边必被舍弃。这就与求最小生成树的“破坏法”完全相符了。

用 Prim 算法求图中的最小生成树，最小生成树上各边的长度之和即是所求的答案。时间复杂度为 $O(n^2)$ 。

但是，本题还有其特殊性。本题是在 Euclid 空间求最小生成树，Euclid 空间最小生成树有 $O(n \log n)$ 的算法，是用 Voronoi 图+Kruskal 算法（或用 Prim+heap 代替 Kruskal）实现的。

5.9 速度限制

源程序名	speed.??? (pas, c, cpp)
可执行文件名	speed.exe
输入文件名	speed.in
输出文件名	speed.out

【问题描述】

在这个繁忙的社会中，我们往往不再去选择最短的道路，而是选择最快的路线。开车时每条道路的限速成为最关键的问题。不幸的是，有一些限速的标志丢失了，因此你无法得知应该开多快。一种可以辩解的解决方案是，按照原来的速度行驶。你的任务是计算两地间的最快路线。

你将获得一份现代化城市的道路交通信息。为了使问题简化，地图只包括路口和道路。每条道路是有向的，只连接了两条道路，并且最多只有一块限速标志，位于路的起点。两地 A 和 B，最多只有一条道路从 A 连接到 B。你可以假设加速能够在瞬间完成并且不会有交通堵塞等情况影响你。当然，你的车速不能超过当前的速度限制。

【输入】

输入文件 SPEED.IN 的第一行是 3 个整数 N, M 和 D ($2 \leq N \leq 150$)，表示道路的数目，用 0..N-1 标记。M 是道路的总数，D 表示你的目的地。接下来的 M 行，每行描述一条道路，每行有 4 个整数 A ($0 \leq A < N$)，B ($0 \leq B < N$)，V ($0 \leq V \leq 500$) 和 L ($1 \leq L \leq 500$)，这条路是从 A 到 B 的，速度限制是 V，长度为 L。如果 V 是 0，表示这条路的限速未知。如果 V 不为 0，则经过该路的时间 $T=L/V$ 。否则 $T=L/V_{old}$ ， V_{old} 是你到达该路口前的速度。开始时你位于 0 点，并且速度为 70。

【输出】

输出文件 SPEED.OUT 仅一行整数，表示从 0 到 D 经过的城市。

输出的顺序必须按照你经过这些城市的顺序，以 0 开始，以 D 结束。仅有一条最快路线。

【样例】

speed.in	speed.out
6 15 1	0 5 2 3 1
0 1 25 68	
0 2 30 50	

0 5 0 101
1 2 70 77
1 3 35 42
2 0 0 22
2 1 40 86
2 3 0 23
2 4 45 40
3 1 64 14
3 5 0 23
4 1 95 8
5 1 0 84
5 2 90 64
5 3 36 40

【问题分析】

首先，利用预处理计算任意两个节点之间只经过无限速标志的路的最短距离。这可以用 Floyd 算法得到，时间复杂度为 $O(n^3)$ 。

计算城市 1 到城市 D 之间最快路径时，只需对 Dijkstra 稍作修改即可：在 Dijkstra 算法中，用一个已计算出最短路径的节点去刷新其他节点当前最短路径长度时，除了要枚举有限速标志的路以外，还要在此路的基础上，枚举通过此路后要经过无限速标志的路到达的节点。时间复杂度为 $O(n^2+mn)$ ，即 $O(mn)$ 。